How Lambda processes records from stream and queue-based event sources

An *event source mapping* is a Lambda resource that reads items from stream and queue-based services and invokes a function with batches of records. Within an event source mapping, resources called *event pollers* actively poll for new messages and invoke functions. By default, Lambda automatically scales event pollers, but for certain event source types, you can use provisioned mode to control the minimum and maximum number of event pollers dedicated to your event source mapping.

The following services use event source mappings to invoke Lambda functions:

- Amazon DocumentDB (with MongoDB compatibility) (Amazon DocumentDB)
- Amazon DynamoDB
- Amazon Kinesis
- Amazon MQ
- Amazon Managed Streaming for Apache Kafka (Amazon MSK)
- Self-managed Apache Kafka
- Amazon Simple Queue Service (Amazon SQS)

> **Warning**
>
> Lambda event source mappings process each event at least once, and duplicate processing of records can occur. To avoid potential issues related to duplicate events, we strongly recommend that you make your function code idempotent. To learn more, see How do I make my Lambda function idempotent in the AWS Knowledge Center.

How event source mappings differ from direct triggers

Some AWS services can directly invoke Lambda functions using *triggers*. These services push events to Lambda, and the function is invoked immediately when the specified event occurs. Triggers are suitable for discrete events and real-time processing. When you create a trigger using the Lambda console, the console interacts with the corresponding AWS service to configure the event notification on that service. The trigger is actually stored and managed by the service that

generates the events, not by Lambda. Here are some examples of services that use triggers to invoke Lambda functions:

- **Amazon Simple Storage Service (Amazon S3):** Invokes a function when an object is created, deleted, or modified in a bucket. For more information, see Tutorial: Using an Amazon S3 trigger to invoke a Lambda function.
- **Amazon Simple Notification Service (Amazon SNS):** Invokes a function when a message is published to an SNS topic. For more information, see Tutorial: Using AWS Lambda with Amazon Simple Notification Service.
- **Amazon API Gateway:** Invokes a function when an API request is made to a specific endpoint. For more information, see Invoking a Lambda function using an Amazon API Gateway endpoint.

Event source mappings are Lambda resources created and managed within the Lambda service. Event source mappings are designed for processing high-volume streaming data or messages from queues. Processing records from a stream or queue in batches is more efficient than processing records individually.

Batching behavior

By default, an event source mapping batches records together into a single payload that Lambda sends to your function. To fine-tune batching behavior, you can configure a batching window (MaximumBatchingWindowInSeconds) and a batch size (BatchSize). A batching window is the maximum amount of time to gather records into a single payload. A batch size is the maximum number of records in a single batch. Lambda invokes your function when one of the following three criteria is met:

- **The batching window reaches its maximum value.** Default batching window behavior varies depending on the specific event source.
  - **For Kinesis, DynamoDB, and Amazon SQS event sources:** The default batching window is 0 seconds. This means that Lambda invokes your function as soon as records are available. To set a batching window, configure `MaximumBatchingWindowInSeconds`. You can set this parameter to any value from 0 to 300 seconds in increments of 1 second. If you configure a batching window, the next window begins as soon as the previous function invocation completes.
  - **For Amazon MSK, self-managed Apache Kafka, Amazon MQ, and Amazon DocumentDB event sources:** The default batching window is 500 ms. You can configure `MaximumBatchingWindowInSeconds` to any value from 0 seconds to 300 seconds in

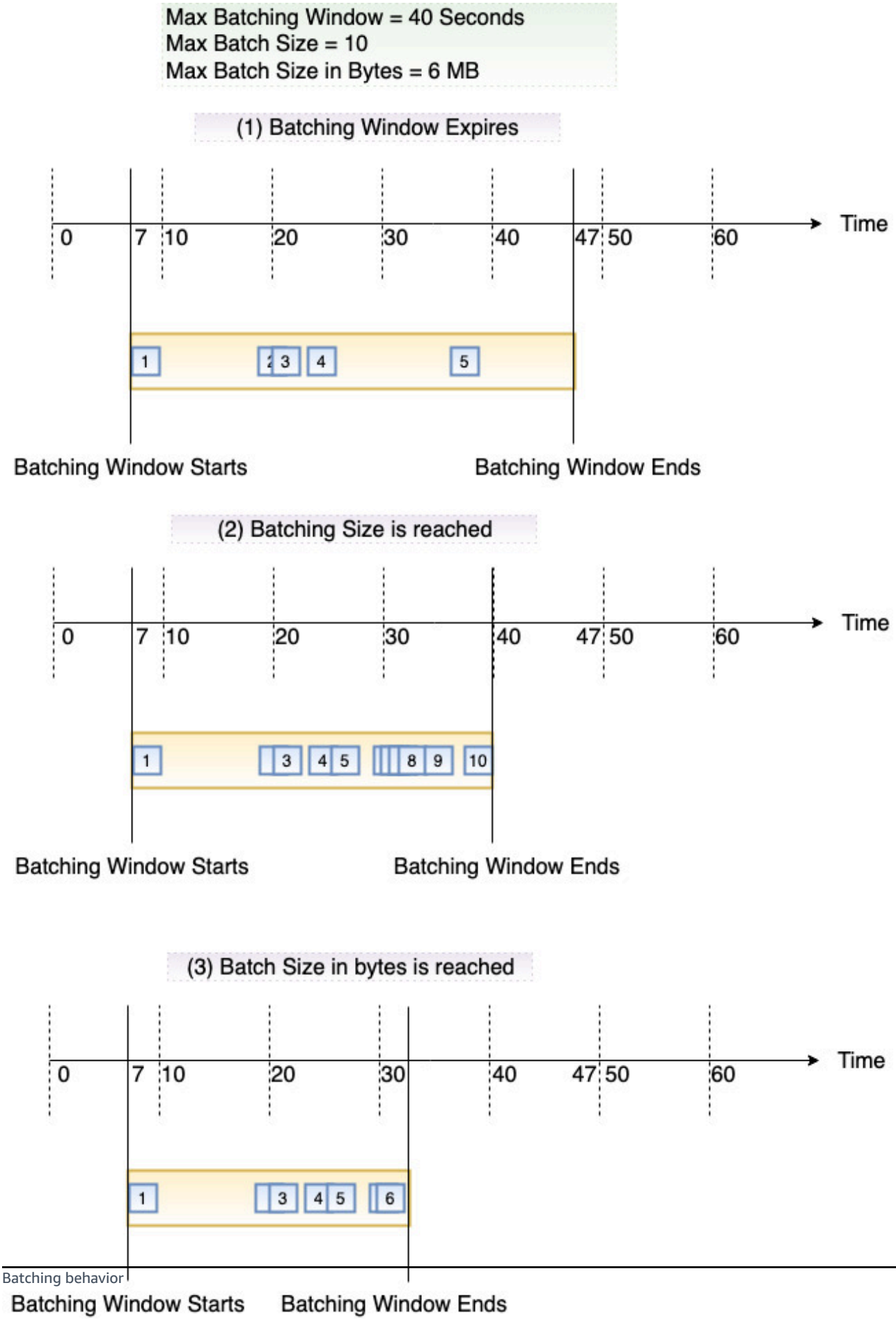
increments of seconds. In provisioned mode for Kafka event source mappings, when you configure a batching window, the next window begins as soon as the previous batch is completed. In non-provisioned Kafka event source mappings, when you configure a batching window, the next window begins as soon as the previous function invocation completes. To minimize latency when using Kafka event source mappings in provisioned mode, set `MaximumBatchingWindowInSeconds` to 0. This setting ensures that Lambda will start processing the next batch immediately after completing the current function invocation. For additional information on low latency processing, see Low latency Apache Kafka.

- **For Amazon MQ and Amazon DocumentDB event sources:** The default batching window is 500 ms. You can configure `MaximumBatchingWindowInSeconds` to any value from 0 seconds to 300 seconds in increments of seconds. A batching window begins as soon as the first record arrives.

> **Note**
>
> Because you can only change `MaximumBatchingWindowInSeconds` in increments of seconds, you cannot revert to the 500 ms default batching window after you have changed it. To restore the default batching window, you must create a new event source mapping.

- **The batch size is met.** The minimum batch size is 1. The default and maximum batch size depend on the event source. For details about these values, see the BatchSize specification for the `CreateEventSourceMapping` API operation.
- **The payload size reaches 6 MB.** You cannot modify this limit.

The following diagram illustrates these three conditions. Suppose a batching window begins at `t = 7` seconds. In the first scenario, the batching window reaches its 40 second maximum at `t = 47` seconds after accumulating 5 records. In the second scenario, the batch size reaches 10 before the batching window expires, so the batching window ends early. In the third scenario, the maximum payload size is reached before the batching window expires, so the batching window ends early.


Max Batching Window = 40 Seconds
Max Batch Size = 10
Max Batch Size in Bytes = 6 MB
(1) Batching Window Expires
0
7
10
20
30
40
47
50
60
Time
1
2
3
4
5
Batching Window Starts
Batching Window Ends
(2) Batching Size is reached
0
7
10
20
30
40
47
50
60
Time
1
3
4
5
8
9
10
Batching Window Starts
Batching Window Ends
(3) Batch Size in bytes is reached
0
7
10
20
30
40
47
50
60
Time
1
3
4
5
6
Batching Window Starts
Batching Window Ends

We recommend that you test with different batch and record sizes so that the polling frequency of each event source is tuned to how quickly your function is able to complete its task. The CreateEventSourceMapping `BatchSize` parameter controls the maximum number of records that can be sent to your function with each invoke. A larger batch size can often more efficiently absorb the invoke overhead across a larger set of records, increasing your throughput.

Lambda doesn't wait for any configured extensions to complete before sending the next batch for processing. In other words, your extensions may continue to run as Lambda processes the next batch of records. This can cause throttling issues if you breach any of your account's concurrency settings or limits. To detect whether this is a potential issue, monitor your functions and check whether you're seeing higher concurrency metrics than expected for your event source mapping. Due to short times in between invokes, Lambda may briefly report higher concurrency usage than the number of shards. This can be true even for Lambda functions without extensions.

By default, if your function returns an error, the event source mapping reprocesses the entire batch until the function succeeds, or the items in the batch expire. To ensure in-order processing, the event source mapping pauses processing for the affected shard until the error is resolved. For stream sources (DynamoDB and Kinesis), you can configure the maximum number of times that Lambda retries when your function returns an error. Service errors or throttles where the batch does not reach your function do not count toward retry attempts. You can also configure the event source mapping to send an invocation record to a destination when it discards an event batch.

Provisioned mode

Lambda event source mappings use event pollers to poll your event source for new messages. By default, Lambda manages the autoscaling of these pollers based on message volume. When message traffic increases, Lambda automatically increases the number of event pollers to handle the load, and reduces them when traffic decreases.

In provisioned mode, you can fine-tune the throughput of your event source mapping by defining minimum and maximum limits for dedicated polling resources that remain ready to handle expected traffic patterns. These resources auto-scale 3 times faster to handle sudden spikes in event traffic and provide 16 times higher capacity to process millions of events. This helps you build highly responsive event-driven workloads with stringent performance requirements.

In Lambda, an event poller is a compute unit with throughput capabilities that vary by event source type. For Amazon MSK and self-managed Apache Kafka, each event poller can handle up to 5 MB/sec of throughput or up to 5 concurrent invocations. For example, if your event source produces

an average payload of 1 MB and the average duration of your function is 1 second, a single Kafka event poller can support 5 MB/sec throughput and 5 concurrent Lambda invocations (assuming no payload transformation). For Amazon SQS, each event poller can handle up to 1 MB/sec of throughput or up to 10 concurrent invocations. Using provisioned mode incurs additional costs based on your event poller usage. For pricing details, see AWS Lambda pricing.

Provisioned mode is available for Amazon MSK, self-managed Apache Kafka, and Amazon SQS event sources. While concurrency settings give you control over the scaling of your function, provisioned mode gives you control over the throughput of your event source mapping. To ensure maximum performance, you might need to adjust both settings independently.

Provisioned mode is ideal for real-time applications requiring consistent event processing latency, such as financial services firms processing market data feeds, e-commerce platforms providing real-time personalized recommendations, and gaming companies managing live player interactions.

Each event poller supports different throughput capacity:

- For Amazon MSK and self-managed Apache Kafka: up to 5 MB/sec of throughput or up to 5 concurrent invokes
- For Amazon SQS: up to 1 MB/sec of throughput or up to 10 concurrent invokes or up to 10 SQS polling API calls per second.

For Amazon SQS event source mappings, you can set the minimum number of pollers between 2 and 200 with a default of 2, and the maximum number between 2 and 2,000 with a default of 200. Lambda scales the number of event pollers between your configured minimum and maximum, quickly adding up to 1,000 concurrency per minute to provide consistent, low-latency processing of your events.

For Kafka event source mappings, you can set the minimum number of pollers between 1 and 200 with a default of 1, and the maximum number between 1 and 2,000 with a default of 200. Lambda scales the number of event pollers between your configured minimum and maximum based on your event backlog in your topic to provide low-latency processing of your events.

Note that for Amazon SQS event sources, the maximum concurrency setting cannot be used with provisioned mode. When using provisioned mode, you control concurrency through the maximum event pollers setting.

For details about configuring provisioned mode, see the following sections:

- Configuring provisioned mode for Amazon MSK event source mappings
- Configuring provisioned mode for self-managed Apache Kafka event source mappings
- Using provisioned mode with Amazon SQS event source mappings

To minimize latency in provisioned mode, set `MaximumBatchingWindowInSeconds` to 0. This setting ensures that Lambda will start processing the next batch immediately after completing the current function invocation. For additional information on low latency processing, see Low latency Apache Kafka.

After configuring provisioned mode, you can observe the usage of event pollers for your workload by monitoring the `ProvisionedPollers` metric. For more information, see the section called "Event source mapping metrics".

Event source mapping API

To manage an event source with the AWS Command Line Interface (AWS CLI) or an AWS SDK, you can use the following API operations:

- CreateEventSourceMapping
- ListEventSourceMappings
- GetEventSourceMapping
- UpdateEventSourceMapping
- DeleteEventSourceMapping

Using tags on event source mappings

You can tag event source mappings to organize and manage your resources. Tags are free-form key-value pairs associated with your resources that are supported across AWS services. For more information about use cases for tags, see Common tagging strategies in the *Tagging AWS Resources and Tag Editor Guide*.

Event source mappings are associated with functions, which can have their own tags. Event source mappings do not automatically inherit tags from functions. You can use the AWS Lambda API to view and update tags. You can also view and update tags while managing a specific event source mapping in the Lambda console, including those using Provisioned Mode for Amazon SQS.

Permissions required for working with tags

To allow an AWS Identity and Access Management (IAM) identity (user, group, or role) to read or set tags on a resource, grant it the corresponding permissions:

- **lambda:ListTags**–When a resource has tags, grant this permission to anyone who needs to call `ListTags` on it. For tagged functions, this permission is also necessary for `GetFunction`.
- **lambda:TagResource**–Grant this permission to anyone who needs to call `TagResource` or perform a tag on create.

Optionally, consider granting the **lambda:UntagResource** permission as well to allow `UntagResource` calls to the resource.

For more information, see Identity-based IAM policies for Lambda.

Using tags with the Lambda console

You can use the Lambda console to create event source mappings that have tags, add tags to existing event source mappings, and filter event source mappings by tag, including those configured in Provisioned Mode for Amazon SQS.

When you add a trigger for supported stream and queue-based services using the Lambda console, Lambda automatically creates an event source mapping. For more information about these event sources, see the section called "Event source mappings". To create an event source mapping in the console, you will need the following prerequisites:

- A function.
- An event source from an affected service.

You can add the tags as part of the same user interface you use to create or update triggers.

To add a tag when you create a event source mapping

1. Open the Functions page of the Lambda console.
2. Choose the name of your function.
3. Under **Function overview**, choose **Add trigger**.
4. Under **Trigger configuration**, in the dropdown list, choose the name of the service your event source comes from.

5. Provide the core configuration for your event source. For more information about configuring your event source, consult the section for the related service in *Integrating other services*.
6. Under **Event source mapping configuration**, choose **Additional settings**.
7. Under **Tags**, choose **Add new tag**
8. In the **Key** field, enter your tag key. For information about tagging restrictions, see Tag naming limits and requirements in the *Tagging AWS Resources and Tag Editor Guide*.
9. Choose **Add**.

To add tags to an existing event source mapping

1. Open Event source mappings in the Lambda console.
2. From the resource list, choose the **UUID** for the event source mapping corresponding to your **Function** and **Event source ARN**.
3. From the tab list below the **General configuration pane**, choose **Tags**.
4. Choose **Manage tags**.
5. Choose **Add new tag**.
6. In the **Key** field, enter your tag key. For information about tagging restrictions, see Tag naming limits and requirements in the *Tagging AWS Resources and Tag Editor Guide*.
7. Choose **Save**.

To filter event source mappings by tag

1. Open Event source mappings in the Lambda console.
2. Choose the search box.
3. From the dropdown list, select your tag key from below the **Tags** subheading.
4. Select **Use: "tag-name"** to see all event source mappings tagged with this key, or choose an **Operator** to further filter by value.
5. Select your tag value to filter by a combination of tag key and value.

The search box also supports searching for tag keys. Enter the name of a key to find it in the list.

Using tags with the AWS CLI

You can add and remove tags on existing Lambda resources, including event source mappings, with the Lambda API. You can also add tags when creating an event source mapping, which allows you to keep a resource tagged through its entire lifecycle.

Updating tags with the Lambda tag APIs

You can add and remove tags for supported Lambda resources through the TagResource and UntagResource API operations.

You can call these operations using the AWS CLI. To add tags to an existing resource, use the `tag-resource` command. This example adds two tags, one with the key *`Department`* and one with the key *`CostCenter`*.

```
aws lambda tag-resource \
--resource arn:aws:lambda:us-east-2:123456789012:resource-type:my-resource \
--tags Department=Marketing,CostCenter=1234ABCD
```

To remove tags, use the `untag-resource` command. This example removes the tag with the key *`Department`*.

```
aws lambda untag-resource --resource arn:aws:lambda:us-east-1:123456789012:resource-
type:resource-identifier \
--tag-keys Department
```

Adding tags when you create an event source mapping

To create a new Lambda event source mapping with tags, use the CreateEventSourceMapping API operation. Specify the `Tags` parameter. You can call this operation with the `create-event-source-mapping` AWS CLI command and the `--tags` option. For more information about the CLI command, see create-event-source-mapping in the *AWS CLI Command Reference*.

Before using the `Tags` parameter with `CreateEventSourceMapping`, ensure that your role has permission to tag resources alongside the usual permissions needed for this operation. For more information about permissions for tagging, see the section called “Permissions required for working with tags”.

Viewing tags with the Lambda tag APIs

To view the tags that are applied to a specific Lambda resource, use the `ListTags` API operation. For more information, see ListTags.

You can call this operation with the `list-tags` AWS CLI command by providing an ARN (Amazon Resource Name).

```
aws lambda list-tags --resource arn:aws:lambda:us-east-1:123456789012:resource-
type:resource-identifier
```

Filtering resources by tag

You can use the AWS Resource Groups Tagging API GetResources API operation to filter your resources by tags. The `GetResources` operation receives up to 10 filters, with each filter containing a tag key and up to 10 tag values. You provide `GetResources` with a `ResourceType` to filter by specific resource types.

You can call this operation using the `get-resources` AWS CLI command. For examples of using `get-resources`, see get-resources in the *AWS CLI Command Reference*.